

Bonus Assignment

EE5178: Modern Computer Vision

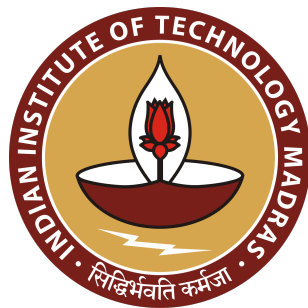
By

Shahid Mohammad Mulla

ME21B037

Course Instructor:

Prof. Kaushik Mitra



Indian Institute of Technology, Madras

April, 2025

Contents

1	Panoramic Stitching	1
1.1	Getting the features	1
1.2	Matching Keypoints and their corresponding descriptors	3
1.3	Finding Homography	5
1.4	RANSAC	7
1.5	Panoramic Stitching	10
	Appendix	14

Chapter 1

Panoramic Stitching

1.1 GETTING THE FEATURES

The following are the two images that have been given to us for panoramic stitching. We can see both the images capture a similar scenery but the camera has moved by some amount to capture different specifics of the scenery.



Figure 1.1: Img 1



Figure 1.2: Img 2

I've used SIFT algorithm to detect the key-points in the two images. SIFT. The algorithm mainly consists of 4 steps:

- **Scale-space Extrema Detection:** Identify keypoints at different scales by detecting local extrema in the Difference-of-Gaussian (DoG) pyramid.
- **Keypoint Localization:** Refine keypoints by accurately localizing their positions and eliminating low-contrast and poorly localized keypoints.

- **Orientation Assignment:** Assign a consistent orientation to each keypoint based on local image gradients, making keypoints invariant to image rotation.
- **Descriptor Generation:** Generate a distinctive descriptor for each keypoint based on local image gradients, ensuring robustness to changes in illumination and view-point.

OpenCV has a **pre-existing** function to perform the same.

Below is the code snippet for performing the same

```

1 def get_features(img):
2     '''
3     Compute SURF/SIFT/ORB features using cv2 library functions. Use default
4     parameters when computing the keypoints.
5     Input:
6         img: cv2 image
7     Returns:
8         keypoints: a list of cv2 keypoints
9         descriptors: a list of feature descriptors
10    '''
11    # =====
12    # TODO
13
14    # Initializing the Sift detector
15    sift = cv2.SIFT_create()
16    # Detecting keypoints and computing descriptors
17    keypoints, descriptors = sift.detectAndCompute(img, None)
18    # =====
19    return keypoints, descriptors

```

Listing 1.1: Code for Getting features using SIFT

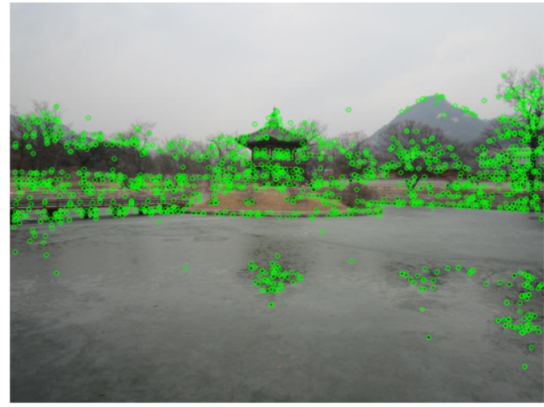
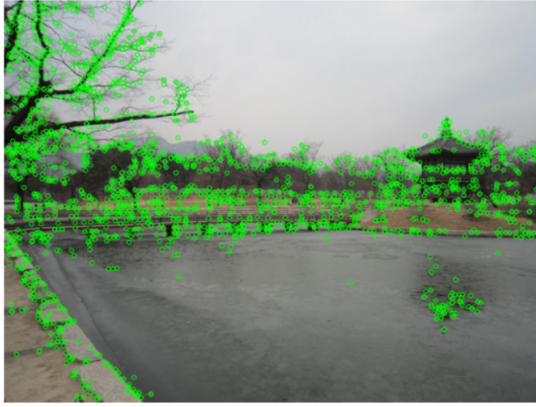


Figure 1.3: Detected features in Img 1 Figure 1.4: Detected features in Img 2

1.2 MATCHING KEYPOINTS AND THEIR CORRESPONDING DESCRIPTORS

In the function `match_keypoints`, the SIFT feature descriptors of two images are compared to find matching keypoints between them. This comparison is performed using the Brute-Force Matcher (BFMatcher) from the OpenCV library. The BFMatcher is configured with the L2 (Euclidean) distance metric to measure the similarity between descriptors. For each keypoint descriptor in the first image (`desc_1`), the function finds the two nearest neighbors in the descriptors of the second image (`desc_2`). If the distance ratio between the closest and second closest matches is less than a specified threshold (defaulted to 0.5), it indicates a good match. These good matches are then returned as a list of feature matches. This matching process helps in identifying corresponding keypoints across images which are then later used to find the homography matrix.

There were a total of 238 good matches between the two images

Below is the code snippet for performing the same

```
1 def match_keypoints(desc_1, desc_2, ratio=0.5):
2     '''
3     You may use cv2 library functions.
4     Input:
5         desc_1, desc_2: list of feature descriptors
6     Return:
7         matches: list of feature matches
8     '''
9     # =====
10    # TODO
11    bf = cv2.BFMatcher(cv2.NORM_L2, crossCheck=False)
12    matches = bf.knnMatch(desc_1, desc_2, k=2) #2 matches
13    good = [] #Selecting just the good matches where the least L2 Norm for a
14              #match is less than 0.75 times the second least L2 norm for the same match.
15    for m,n in matches:
16        if m.distance < ratio*n.distance:
17            good.append([m])
18    return good
19    # =====
```

Listing 1.2: Code for Matching the features in the given images

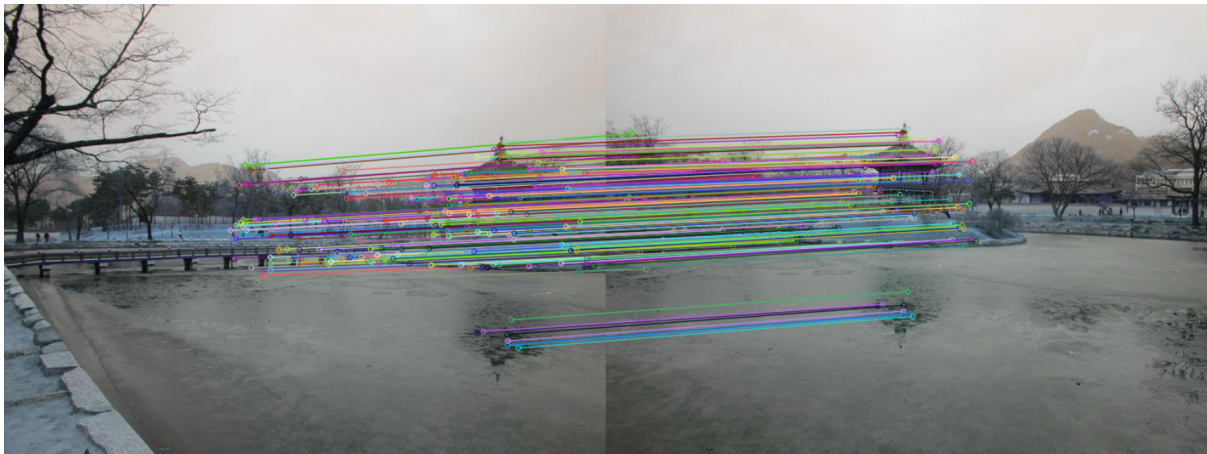


Figure 1.5: Matching keypoints in the two images

1.3 FINDING HOMOGRAPHY

The `find_homography` function implements the Direct Linear Transform (DLT) method to estimate the homography matrix ($\mathbf{H}$) that represents the transformation mapping from one set of 2D points ($\mathbf{pts_1}$) to another set ($\mathbf{pts_2}$). The function constructs a set of linear equations by stacking the constraints imposed by corresponding points in both sets. These constraints are expressed in the form of homogeneous coordinates and assembled into a matrix $\mathbf{A}$. The system of equations $\mathbf{A} * \mathbf{h} = \mathbf{0}$ is then solved using the Singular Value Decomposition (SVD) method, where $\mathbf{h}$ is a vectorized representation of the elements of $\mathbf{H}$. By taking the last column of the right singular vectors matrix $\mathbf{V}$ obtained from SVD, the homography matrix $\mathbf{H}$ is reconstructed. Finally, the matrix $\mathbf{H}$ is normalized by dividing it by its last element to ensure that it satisfies the homogenous coordinate property. The resulting homography matrix $\mathbf{H}$ represents the geometric transformation between the two sets of points.

Below is the code snippet for performing the same

```
1 def find_homography(pts_1, pts_2):
2     '''
3     Implement Direct Linear Transform to find a homography that estimates the
4     transformation mapping from pts_1 to pts_2.
5     e.g. If x is in pts_1 and y is in pts_2, then  $y = H * x$ 
6     Input:
7         pts_1, pts_2: (N, 2) matrix
8     Return:
9         H: the resultant homography matrix (3 x 3)
10    '''
11    # =====
12    # TODO
13
14    A = []
15    for i in range(pts_1.shape[0]):
16        x1, y1 = pts_1[i]
17        x2, y2 = pts_2[i]
18        A.append([-x1, -y1, -1, 0, 0, 0, x2 * x1, x2 * y1, x2])
19        A.append([0, 0, 0, -x1, -y1, -1, y2 * x1, y2 * y1, y2])
20    A = np.array(A)
21
22    # Solve the system using least squares
23    _, _, V = np.linalg.svd(A) #Using SVD to compute V and using that to get the
24    # Homography Matrix.
25    H = V[-1].reshape(3, 3)
26    H = H/H[2, 2]
27
28    # =====
29    return H
```

Listing 1.3: Code for computing Homography matrix

1.4 RANSAC

The `transform_ransac` function implements the **RANSAC** algorithm to estimate the homography matrix (**H**) that best represents the transformation mapping between two sets of 2D points (**pts_1** and **pts_2**). In each iteration of RANSAC, a random sample of four point correspondences is selected from the sets of points, and a candidate homography matrix (**H**) is computed using the `find_homography` function. Then, the number of inliers is calculated by transforming each point in **pts_1** to its corresponding location in **pts_2** using **H**, and comparing the transformed point with the actual point in **pts_2**. If the Euclidean distance between the transformed point and the actual point is less than a predefined threshold (*threshold*), the point is considered an inlier. The homography matrix with the highest number of inliers is selected as the best model. This process is repeated for a specified number of iterations. I have set the `num_`iterations to be 2000 to find the optimal homography matrix that aligns the two sets of points. Finally, the best homography matrix is returned as the output of the function.

There were a total of 109 Inliers for the best homography matrix found after 200 iterations. The following came out as the homography matrix using the above function:

$$\begin{bmatrix} 1.70804158e+00 & -6.78090077e-02 & -4.73796897e+02 \\ 2.56842592e-01 & 1.42366017e+00 & -1.60528660e+02 \\ 9.54604068e-04 & -1.20738094e-04 & 1.00000000e+00 \end{bmatrix}$$

Below is the code snippet for performing the same

```
1 def transform_ransac(pts_1, pts_2, num_iterations=2000, threshold=0.5):
2     '''
3     Implement RANSAC to estimate homography matrix.
4     Input:
5         pts_1, pts_2: (N, 2) matrices
6         num_iterations: number of RANSAC iterations
7         threshold: distance threshold to consider inliers
8     Return:
9         best_model: homography matrix with most inliers
10    '''
11    best_inliers = 0
12    best_model = None
13
14    for _ in tqdm(range(num_iterations)):
15        # Selecting 4 random points from the set of good matches we have to
16        # compute the homography matrix in accordance
17        indices = np.random.choice(pts_1.shape[0], 4, replace=False)
18        src_pts = pts_1[indices]
19        dst_pts = pts_2[indices]
20
21        # Computing the homography matrix
22        H = find_homography(src_pts, dst_pts)
23
24        # Calculating inliers
25        inliers = 0
26        for i in range(pts_1.shape[0]):
27            x1, y1 = pts_1[i]
28            x2, y2 = pts_2[i]
29            p1 = np.array([x1, y1, 1]).reshape(3, 1)
30            p2 = np.array([x2, y2, 1]).reshape(3, 1)
31            p2_est = np.dot(H, p1)
32            p2_est /= p2_est[2]
33
34            if np.linalg.norm(p2[:2] - p2_est[:2]) < threshold: #We set a
35                # threshold to compare the difference between H*p1 and p2 since multiplying to
36                # H might not lead to an exact pixel in the other image leading to slight
37                # inaccuracies, so we keep a threshold to account for the same.
```

```

33         inliers += 1
34
35     # Updating best model if current model has more inliers
36     if inliers > best_inliers:
37         best_inliers = inliers
38         best_model = H
39
40     print(f"inliers/matches: {best_inliers}/{len(matches)}")
41     return best_model

```

Listing 1.4: Code for performing RANSAC operation to get appropriate homography matrix

1.5 PANORAMIC STITCHING

The `panoramic_stitching` function performs panoramic image stitching by aligning two input images (**left** and **right**) using feature matching and homography estimation. Firstly, both input images are normalized to floating-point values between 0 and 1 using the `cv2.normalize` function. Keypoints and their corresponding descriptors are then extracted from each image using the `get_features` function. Next, feature matching is performed to find correspondences between keypoints in the two images using the `match_keypoints` function.

The RANSAC algorithm is then employed to estimate the homography matrix (**H**) that best represents the geometric transformation between the matched keypoints. In each iteration of RANSAC, a random sample of four correspondences is selected, and a candidate homography matrix is computed using the `transform_ransac` function. The homography matrix is used to warp the left image to align with the right image, taking into account both the homography and translation matrices.

To ensure that both images are properly aligned, a translation matrix is computed to shift the bottom corner of the warped left image to the origin and to adjust the position of the right image relative to the left image. The resulting images are then blended together by iteratively averaging pixel intensities from the warped left and right images. Finally, the stitched image is cropped to remove any black borders resulting from the warping process, and the stitched image is returned as the output of the function.

Below is the code snippet for performing the same

```
1 def panoramic_stitching(left, right):
2     left = cv2.normalize(left.astype('float'), None,
3                           0.0, 1.0, cv2.NORM_MINMAX)
4
5     right = cv2.normalize(right.astype('float'), None,
6                           0.0, 1.0, cv2.NORM_MINMAX)
7
8     # Detecting keypoints and computing descriptors for both images
```

```

9     kp_1, desc_1 = get_features(img1) #Extracting keypoints and corresponding
descriptors from image 1
10    kp_2, desc_2 = get_features(img2) #Extracting keypoints and corresponding
descriptors from image 2
11    good_matches = match_keypoints(desc_1, desc_2)
12
13    # Extracting corresponding keypoints
14    src_pts = np.float32([kp_1[m[0].queryIdx].pt for m in good_matches])
15    dst_pts = np.float32([kp_2[m[0].trainIdx].pt for m in good_matches])
16
17    # Using RANSAC to estimate homography matrix
18    H = transform_ransac(src_pts, dst_pts)
19
20    # left image
21    height_1, width_1, channel_1 = left.shape
22    corners = [[0, 0, 1], [width_1, 0, 1], [width_1, height_1, 1], [0, height_1
, 1]] #The corners for the first image.
23    corners_new = [np.dot(H, corner) for corner in corners] #Computing new
corners based on the computed homography matrix given by RANSAC
24    corners_new = np.array(corners_new).T
25    x_news = corners_new[0] / corners_new[2] #Normalization
26    y_news = corners_new[1] / corners_new[2] #Normalization
27    y_min = min(y_news)
28    x_min = min(x_news)
29
30    translation_mat = np.array([[1, 0, -x_min], [0, 1, -y_min], [0, 0, 1]]) #
translation matrix to shift the bottom corner of the transformed image to
the origin and to
31    H = np.dot(translation_mat, H) #
the other image wrt the first image
32
33    x_max = max(x_news)
34    y_max = max(y_news)
35    # Getting height, width
36    height_new = int(round(abs(y_min) + (y_max - y_min)))
37    width_new = int(round(abs(x_min) + (x_max - x_min)))
38    size = (width_new, height_new)
39

```

```

40     # right image
41     warped_l = cv2.warpPerspective(src=left, M=M, dsize=size) #Warping the
first image taking into account both the homography and the translation
matrix
42
43     height_r, width_r, channel_r = right.shape
44
45     height_new = int(round(abs(y_min) + height_r))
46     width_new = int(round(abs(x_min) + width_r))
47     size = (width_new, height_new)
48
49     warped_r = cv2.warpPerspective(src=right, M=translation_mat, dsize=size) #
Warping the second image taking into account the translation matrix
50
51     black = np.zeros(3) # Black pixel.
52     # Stitching procedure, storing results in warped_l.
53     for i in tqdm(range(warped_r.shape[0])): #Putting the two warped images
together by iterating over all pixels in the final image and using warped
image pixel intensities to
54         for j in range(warped_r.shape[1]): #fill the new image
55             pixel_l = warped_l[i, j, :]
56             pixel_r = warped_r[i, j, :]
57             if not np.array_equal(pixel_l, black) and np.array_equal(pixel_r,
black):
58                 warped_l[i, j, :] = pixel_l
59             elif np.array_equal(pixel_l, black) and not np.array_equal(pixel_r,
black):
60                 warped_l[i, j, :] = pixel_r
61             elif not np.array_equal(pixel_l, black) and not np.array_equal(
pixel_r, black):
62                 warped_l[i, j, :] = (pixel_l + pixel_r) / 2
63             else:
64                 pass
65
66     stitch_image = warped_l[:warped_r.shape[0], :warped_r.shape[1], :]
67     return stitch_image

```

Listing 1.5: Code for Stitching the images

The final function i.e. `panoramic_stitching` took 25 seconds to stitch the two images.

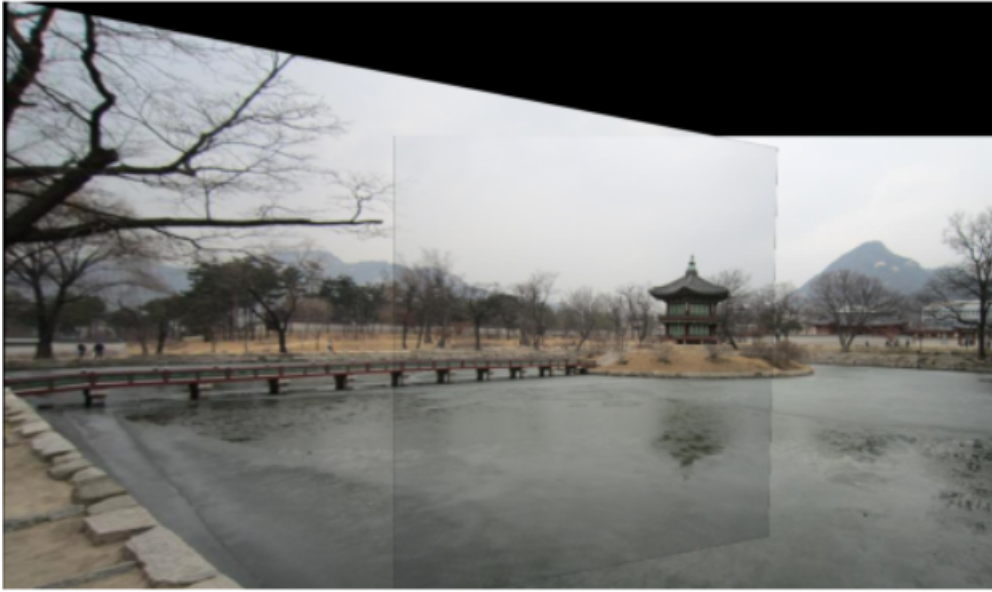


Figure 1.6: Final Stitched Image

Appendix

BLENDING THE TWO IMAGES

Since the illumination in two views cannot be guaranteed to be identical, stitching two images sometimes creates a visible seam. To counter this, we blend the two images using a smooth weighted function. This blends the image edges really well and returns a smooth scenery. We can modify the existing stitch function:

```
1 for i in tqdm(range(warped_r.shape[0])): #Putting the two warped images
    together by iterating over all pixels in the final image and using warped
    image pixel intensities to
2     for j in range(warped_r.shape[1]): #fill the new image
3         pixel_l = warped_l[i, j, :]
4         pixel_r = warped_r[i, j, :]
5         if not np.array_equal(pixel_l, black) and np.array_equal(pixel_r,
black):
6             warped_l[i, j, :] = pixel_l
7         elif np.array_equal(pixel_l, black) and not np.array_equal(pixel_r,
black):
8             warped_l[i, j, :] = pixel_r
9         elif not np.array_equal(pixel_l, black) and not np.array_equal(
pixel_r, black):
10            boundary_distance = np.abs(j - warped_r.shape[1] / 2)
11            weight = gaussian(boundary_distance)
12            smooth_weighted_pixel = (weight * pixel_l + (1 - weight) *
pixel_r)
13            warped_l[i, j, :] = smooth_weighted_pixel
14        else:
15            pass
```

Listing 1.6: Modified code for smoothing